FasterBERT: Speeding Up BERT Inference via Block-Wise Bypasses and Divergence-Based Early Exiting

Anonymous submission

Abstract

Early exiting can reduce the average latency of pre-trained language models (PLMs) via its adaptive inference mechanism and work with other inference speed-up methods like model pruning, thus drawing much attention from the industry. In this work, we propose a novel framework, FasterBERT, which consists of two off-the-shelf methods for improving PLMs' early exiting. We first address the issues of training a multi-exit PLM, the backbone model for early exiting. We propose the novel architecture of block-wise bypasses, which can alleviate the conflicts in jointly training multiple intermediate classifiers and thus improve the overall performances of multi-exit PLM while introducing negligible additional flops to the model. Second, we propose a novel divergence-based early exiting (DIVE) mechanism, which obtains early exiting signals by comparing the predicted distributions of two adjacent layers' exits. Extensive experiments on the GLUE benchmark result in the following takeaways: (a) our block-wise bypass method can effectively improve the cross-layer average performances of a multi-exit PLM and provide a stronger backbone for early exiting. (b) our DIVE method consistently outperforms the baseline early exiting methods under different speed-up ratios. We also conduct ablation studies to demonstrate that our design of block-wise bypass is necessary and does not achieve its superior performances by merely adding more parameters.[1]

Introduction

Since BERT (Devlin et al. 2019), the pre-trained language models (PLMs) have become the default state-of-the-art (SOTA) models for natural language processing (NLP). The recent years have witnessed the rise of many PLMs, such as GPT (Radford et al. 2019), XLNet (Yang et al. 2019), and ALBERT (Lan et al. 2020), and so forth. These BERT-style models achieved considerable improvements in many Natural Language Processing (NLP) tasks by pre-training on the unlabeled corpus and fine-tuning on labeled tasks, such as text classification, natural language inference (NLI), named entity recognition (NER). Despite their outstanding performances, their industrial usage is still limited by the high latency during inference (Tambe et al. 2020; Zhu 2021a). In addition, a special feature of consumer queries is that there are time intervals when the number of queries is exceptionally high. For example, food search engines will be used more often during dinner hours than usual. Thus, deployed pretrained models need to adjust their latency dynamically.

A branch of literature focuses on making PLMs' inference more efficient via adaptive inference (Zhou et al. 2020; Xin et al. 2020; Liu et al. 2020). The idea of adaptive inference is to process simple examples with only shallow layers of BERT and more difficult queries with deeper layers, thus significantly speeding up the inference time on average while maintaining high accuracy. Early exiting is one of the most important adaptive inference methods (Bolukbasi et al. 2017). As depicted in Figure 1, it implements adaptive inference by installing an early exit, i.e., an intermediate prediction layer, at each layer of PLM(multi-exit PLM) and early exiting "easy" samples to speed up inference. All the exits are jointly optimized at the training stage with BERT's parameters. At the inference stage, an early exiting strategy is designed to decide whether to exit at each layer given the currently obtained predictions (from previous and current layers) (Teerapittayanon, McDanel, and Kung 2016; Kaya, Hong, and Dumitras 2019; Xin et al. 2020; Zhou et al. 2020). In this mode, different samples can exit at different depths. The average speedup ratio can be easily controlled with certain hyper-parameters without re-deploying the model services or maintaining a group of models.

In this work, we propose a novel framework, FasterBERT, to improve the early exiting performances of PLMs. FasterBERT consists of two modifications to the current early exiting literature. First, we propose to add a block-wise bypass to each transformer block so that two different representations can be produced, one for the current layer's exit and the other passed to the next transformer block. With this mechanism, an intermediate transformer block will not be distracted by the conflicting tasks in multi-exit BERT training, thus providing high-quality representations for the subsequent layers. As a result, the overall performance of the multi-exit BERT will improve. Second, we propose a novel divergence-based early exiting method (DIVE), which is effective in mining early exiting signals by comparing the predicted probability distributions of adjacent layers.

Extensive experiments and ablation studies are conducted on the GLUE benchmark datasets. The experimental results show that: (a) training multi-exit PLM with block-wise bypasses consistently performs better than the previous SOTA

[1]Code will be made publicly available upon acceptance.

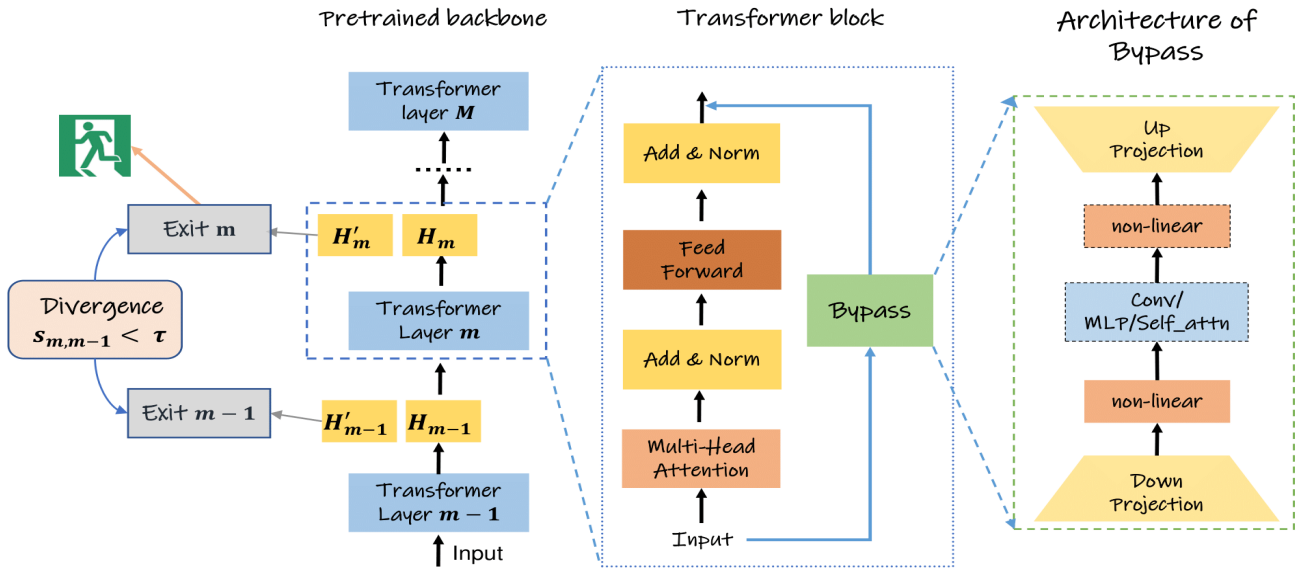


Figure 1: The overview of our FasterBERT framework.

multi-exit model training methods, thus providing a better backbone for early exiting. (b) we show that, with the same multi-exit PLM backbone, our DIVE method can provide better efficiency-performance trade-offs than the previous SOTA early exiting methods. Thus, with our framework, FasterBERT, a PLM can achieve significantly better early exiting performances.

The main contributions of our FasterBERT framework are two-fold:

- We propose a novel method, block-wise bypass, to improve the training of multi-exit PLMs.
- We propose a novel divergence-based early exiting method, DIVE, which outperforms the previous SOTA early exiting methods.

Related Work

Multi-Exit model training

Although the literature has thoroughly investigated the dynamic early exiting mechanisms, the training methods for multi-exit BERT have been relatively less well studied. Joint training (Zhou et al. 2020; Teerapittayanon, McDanel, and Kung 2016; Zhu 2021b; Zhu et al. 2021) fine-tunes the PLM backbone and exits simultaneously. Two-stage training (Liu et al. 2020; Xin et al. 2020) divides the training process into two stages: Stage 1 for fine-tuning the backbone and the last exit, and Stage 2 for training intermediate exits on the fixed backbone. 2ST can conveniently work with knowledge distillation (Liu et al. 2020). Alternating training (Xin et al. 2021) combines the above two methods in an alternating fashion. To improve the performance of joint training, Li et al. (2019) takes a deep look into the conflicts of multi-task learning in joint learning and proposes the gradient equilibrium technique to adjust the gradients from learning different exits. Global-Past-Future (Liao et al. 2021) incorporates the semantic features of all the previous layers to make predictions on the current layer's exit. Sun et al. (2021) adds a regularization term to encourage the intermediate exits to produce diverse predictions.

Our work complements the literature by proposing a novel block-wise bypass method to improve multi-exit PLM training.

Dynamic early exiting mechanisms

There are mainly three early exiting strategies for BERT dynamic exiting. The first one is score-based early exiting (Teerapittayanon, McDanel, and Kung 2016; Liu et al. 2020; Xin et al. 2020; Kaya, Hong, and Dumitras 2019; Schwartz et al. 2020; Zhang et al. 2022). The idea is to calculate the entropy or the maximum probability mass of the predicted probability distribution as an estimation for the confidence of exiting classifiers to enable dynamic early exiting. The second type is the learned exiting (Elbayad et al. 2020; Xin et al. 2021; Li et al. 2021). In this line of work, the early exiting signal is generated by a learnable module in the neural network. For example, BERxiT (Xin et al. 2021) installs a fully connected layer right after each transformer block of BERT to output a score that is used to decide whether the BERT should stop inference and exit early. The third type is the patience-based early exiting, which relies on cross-layer comparison to formulate the exiting signal. PABEE (Zhou et al. 2020) proposes a dynamic exiting strategy analogous to the early stopping method in model training. If the exits' predictions remain unchanged for a pre-defined number of times (patience), the model will stop inference and exit early. Sun et al. (2021) extends the patience-based early exiting mechanism by considering the number of votes from the previous layer supporting the current layer's exit as the early exiting signal.

Our work complements the literature by proposing a novel early exiting method that achieves better quality-efficiency trade-offs.

Preliminaries

In this section, we introduce the necessary background for BERT early exiting. we consider the case of multi-class classification with K classes, $\mathcal{K} = \{1, 2, ..., K\}$. During training, the model will receive a batch $\{(x_i, y_i), i \in \mathcal{I} = \{1, 2, ..., N\}\}$ containing N samples, where $x_i \in \mathbf{R}^L$ is the input sentence consisting of L words, and $y_i \in \mathcal{K}$ is the label.

Early exiting

Early exiting is based on multi-exit PLMs, which are PLMs with classifiers (or exits) at each layer. With M layers, M classifiers $f_m(x; \theta_m) : \mathcal{X} \to \Delta^K (m = 1, 2, ...M)$ are designated at M layers of BERT, each of which maps its input to a probability distribution $p_m \in \Delta^K$, where Δ^K is the probability simplex. $f_m(x; \theta_m)$ can take the form of a simple linear layer (denoted as linear exit) following (Zhou et al. 2020), or an additional multi-head self-attention layer followed by a pooling operation and a linear projection layer (Liu et al. 2020) (denoted as MHA exit).

Multi-exit PLM training Early exiting requires the training of a multi-exit PLM backbone, and the overall performance of such a multi-exit PLM will directly influence the early exiting performances. In this work, we mainly adopt the joint training method (Teerapittayanon, McDanel, and Kung 2016; Zhou et al. 2020) as our basis.[2] Joint training (JT) is the most straightforward fine-tuning strategy for multi-exit PLM training. It minimizes the sum of all classifiers' loss functions and jointly updates all parameters in the process. Formally, the loss function of JT is:

$$\mathcal{L}_{JT} = \sum_{m=1}^{M} \mathcal{L}_m^{CE} \tag{1}$$

where $\mathcal{L}_m^{CE} = \mathcal{L}_m^{CE}(y, f_m(x; \theta_m))$ denotes the cross-entropy loss of the m-th exit.

Inference At inference, the multi-exit PLM can operate in two different modes, depending on whether or not the computational budget to classify an example is known.

Static Early Exiting. If the computational budget is known, we can directly appoint a suitable exit m^* of BERT, $f^{(m^*)}$, to predict all queries.

Dynamic early exiting. Under this mode, after receiving a query input x, the model starts to predict on the classifiers $f^{(1)}$, $f^{(2)}$, ..., in turn in a forward pass, reusing computation where possible. It will continue until it receives a signal to stop early at an exit $m^* < M$, or arrives at the last exit M. At this point, it will output the final predictions based on the current and previous predictions. Note that different samples might exit at different layers under this early exit setting.

[2]Readers are referred to the Appendix for detailed introductions of two-stage training (Liu et al. 2020; Xin et al. 2020) and alternating training (Xin et al. 2021).

Speedup ratio Following PABEE (Zhou et al. 2020; Xin et al. 2021), we mainly report the speedup ratio as the efficiency metric. For each test sample x_i, the early exiting layer is m_i, then the average speedup ratio on the test set is calculated by

$$\text{Speedup} = 1 - \frac{\sum_1^N m_i}{\sum_1^N M}. \tag{2}$$

We choose this efficiency metric for the following reason: (1) it is linear w.r.t. the actual amount of computation; (2) according to our experiments, it is proportional to actual wall-clock runtime and is also more stable across different runs compared with actual runtime due to randomness.

Block-wise bypasses

We now introduce the novel block-wise bypass, which is beneficial for improving the overall performance of multi-exit PLM.

Motivation

Training a multi-exit BERT model requires training multiple exits of different depths simultaneously. Thus, under this setting, an intermediate layer has to fulfill two tasks at once: (a) providing semantic representations to the next layer and (b) providing proper sentence features to the current layer's exit. One may wonder whether these two tasks conflict and result in poor optimizations. Li et al. (2019) investigate this problem and find that each layer's optimizations are often in conflict and can cause optimization instability. They provide a solution called gradient equilibrium, which is to adjust the gradients from each exit's learning objective. However, in our experiments, we will show that this method does not provide significant improvements.

Another solution is to use different sub-networks for these two tasks, following the literature on sparse multi-task learning (Sun et al. 2020). However, this method provides two different representations with two forward passes, each going through a different sub-network, significantly slowing down the inference speed. Thus, this approach does not meet our purpose.

To summarize, we need a novel method that can provide two different representations, one for the next layer and the other for the current layer's prediction, within a single forward pass.

Bypass architectures

We now present the core of our novel FasterBERT framework: the block-wise bypasses (depicted in Figure 1). Denote the representations of the input sentence from the previous transformer block as H_{m-1}. Initially, H_{m-1} will go through the current transformer block, first the multi-head self-attention (MHSA) module with the residue to become $H_{m,MHA}$, then the positional feed-forward (FFN) module with residue, to become $H_{m,F}$, and finally a LayerNorm operation to output H_m.

We would like to employ an efficient bypass B_m to adjust the current layer's representations to fit the task better. B_m is

simple in architecture (On the right side of Figure 1). Upon receiving the input H_{m-1}, B_m down-projects it to $H^{(1)}_{m,B}$, from dimension d to dimension r (where $r << d$) using a down-projection matrix $W_{down} \in \mathbb{R}^{d \times r}$. The $H^{(1)}_{m,B}$ will go through a non-linear activation function g_1, a neural network encoding operation (denoted as Enc) and another activation function g_2, and become $H^{(2)}_{m,B}$. $H^{(2)}_{m,B}$ will then be up-projected to $H^{(3)}_{m,B}$ with dimension d, by an up-projection matrix $W_{up} \in \mathbb{R}^{r \times d}$. We refer to r as the bottleneck dimension. Formally, B_m can be expressed as:

$$H^{(3)}_{m,B} \leftarrow g_2(\text{Enc}(g_1(H_{m-1}W_{down})))W_{up}. \quad (3)$$

g_1 and g_2 usually are non-linear activation functions like GELU (Hendrycks and Gimpel 2016), ReLU (Fukushima 2004), SWISH (Ramachandran, Zoph, and Le 2017), and Tanh (Hochreiter and Schmidhuber 1997), or an identity transformation (Identity). The encoder architecture for Enc we consider are chosen from the following commonly used neural network operations: (a) a fully-connected layer (FC-bypass); (b) a single self-attention layer following (Vaswani et al. 2017) (Attn-bypass); (c) a 1-d convolutional layer (Kim 2014) (Conv-bypass); (d) the simple case where the Enc reduces to the identity mapping (Simple-bypass).

Finally, the current transformer block will output two representations, H_m, which is the original intermediate representation, and $H^{'}_m$, which is modified by the bypass, via:

$$\begin{aligned} H^{'}_m &= \text{LayerNorm}(H^{(3)}_{m,B} + H_{m,F}), \\ H_m &= \text{LayerNorm}(H_{m,F}). \end{aligned} \quad (4)$$

H_m will be passed to the following transformer block as input, and $H^{'}_m$ is the modified representation which will be the input of the current layer's exit. We rely on the bypass B_m to extract task-specific representations to facilitate the learning of the exit. Note that the introduction of B_m will not bring too much extra computation since (a) the two representations H_m and $H^{'}_m$ can be calculated in a single forward pass; (b) the bottleneck dimension r can be very small, like 16, so that the added parameters or flops are negligible compared to the pre-trained backbone.

Our work is inspired by the recent work in parameter efficient tuning (He et al. 2021). However, our work is different in the following three aspects: (a) our work deals with the training of multi-exit BERT, not the parameter efficient tuning settings. (b) Our work introduces the block-wise bypasses, while He et al. (2021) only considers the bypasses around the feed-forward layer or the self-attention layer of a transformer block. We will show in our experiments that block-wise bypasses perform better. (c) We add encoding operations inside the bypasses, which proves to be beneficial.

Divergence-based Early Exiting

Motivation

In the existing early exiting literature, the score-based early exiting (Teerapittayanon, McDanel, and Kung 2016; Kaya, Hong, and Dumitras 2019) makes the early exiting decisions based on the confidence score from a single layer. Their early exiting performances might be negatively affected by the over-confidence phenomenon observed in neural networks. The patience-based mechanism (Zhou et al. 2020) validates the early exiting decisions among the previous layers, providing a promising direction.

However, there are two drawbacks in PABEE. First, the early exiting condition in PABEE is coarse. It compares whether a group of consecutive layers predicted the same label. Their motivation is that if the recent layers predicted the same label, the model is confident in this answer and is not likely to change its answer in the remaining layers. However, even if the adjacent layers give the same label, their output distributions might differ. Thus, PABEE might exit early even when the layers have not reached a consensus in the predictions. In addition, It is common for BERT to change its predictions after a few intermediate layers. In Figure 1 of the Appendix, we plot the number of layers at which the multi-exit ALBERT first changes its predicted label on the RTE task of the GLUE benchmark. A fraction of samples on which the multi-exit ALBERT changes its predictions after 3 to 8 layers. Thus, PABEE's early exiting performances with low patience parameters may not be reliable.

To summarize, we need a more fine-grained criterion to generate more reliable early exiting signals.

Our proposed method

We now introduce our divergence-based early exiting method, DIVE. The inference procedure is illustrated in Figure 1. Assume the forward pass has reached layer m. We denote the divergence score between the prediction results of layer m and layer $m^{'}$ ($m > m^{'}$) as $s_{m,m'} = S(p_m, p_{m'}) \in \mathbf{R}$, where $S(\cdot, \cdot)$ is the divergence measure of two probability distributions. The smaller the value of $s_{m,m'}$, the predicted distributions p_m and $p_{m'}$ are more consistent with each other. We use a counter cnt to store the number of times the divergence scores between adjacent layers are less than the pre-defined exiting threshold τ. At layer m, cnt_m is calculated as:

$$cnt_m = \begin{cases} cnt_{m-1} + 1, & \text{if } s_{m,m-1} < \tau, \\ 0, & \text{otherwise.} \end{cases} \quad (5)$$

If $s_{m,m-1}$ is less than the pre-defined threshold, then the patience counter is increased by 1. Otherwise, the patience counter is reset to 0. If cnt_m reaches the pre-defined patience value t, the model stop inference and exits early. Otherwise, the model goes to the next layer. However, if the model does not exit early at intermediate layers, the model uses the final classifier f_M for prediction.

Note that the above-described framework is flexible and general, wrapping the patience-based method (Zhou et al. 2020) as a special case. In Zhou et al. (2020), the value of $s_{i,i-1}$ is 0 only when the labels predicted by two adjacent layers are identical; otherwise, it is set to 1.

Unless stated otherwise, we will mainly use the knowledge distillation objective (Hinton, Vinyals, and Dean 2015)

as the divergence measures:[3]

$$S(p_i, p_j) = \mathcal{L}_{KD}(p_i, p_j) = -\sum_{k=1}^{K} p_i(k) \log(p_j(k)). \quad (6)$$

Experiments

Datasets

We evaluate our proposed approach to the classification tasks on GLUE benchmark (Wang et al. 2018). We only exclude the STS-B task since it is a regression task, and we exclude the WNLI task following previous work (Jiao et al. 2020; Xu et al. 2020). The size of the datasets is shown in the Table 1 of the Appendix.

Baseline methods

For multi-exiting BERT training, we compare our FasterBERT framework with the following baselines: (a) joint training (JT) (Zhou et al. 2020; Teerapittayanon, McDanel, and Kung 2016); (b) two-stage training (2ST) (Liu et al. 2020; Xin et al. 2020); (c) alternating training (ALT) (Xin et al. 2021); (d) Gradient equilibrium (GradEquil) (Li et al. 2019), which incorporates JT with gradient adjustments; (e) Sparse Multi-task (Sparse-MT) (Sun et al. 2020), which is not suitable for model inference speedup but is included here for performance comparisons; (f) fine-tuning ALBERT with a single exit of different depths separately (Single-exits).

We compare the early exiting performances of our DIVE method with the following early exiting methods: (a) Entropy-based method (Entropy) originated from (Teerapittayanon, McDanel, and Kung 2016); (b) Maximum probability based method (Maxprob) (Schwartz et al. 2020); (c) Patience-based method (Patience) (Zhou et al. 2020); (d) learning-to-exit based method (LTE) proposed by Xin et al. (2021).

Exprimental settings

Backbone models We mainly use the open-sourced ALBERT-base (Lan et al. 2020) as the backbone. We also adopt the BERT-base (Devlin et al. 2019)[4] and ElasticBERT (Liu et al. 2022) as the backbone models in the ablation studies.

Training We add a classification layer after each intermediate layer of the PLM as the exits. The exit is a simple linear layer by default, but we also consider the MHA exit suggested by Liu et al. (2020). We fine-tune the multi-exit BERT with one of the four bypasses architectures (in Section) or without bypasses, and the bottleneck dimension r is set as 16. We fine-tune models for at most 25 epochs with an Adam optimizer and warm-up. The warm-up steps are set to be 10% of the optimization steps. Early stopping with patience eight is performed, and the best checkpoint is selected based on the dev set performances. Since we are training multi-exit models in which each layer's exit have a performance score, we mainly focus on improving S-avg, which denotes the cross-layer average score for a given metric S. We also report S-best, the best score among all the layers for the given metric S. We perform grid search for the following hyper-parameter search space: the {GELU, ReLU, SWISH, Tanh, Identity} for the activation functions in the bypasses; {16, 32, 64, 128, 512, 1024} for the batch size; {1e-5, 2e-5, 3e-5, 5e-5} for the learning rate. We implement our FasterBERT and all the baselines on the base of Hugging Face's Transformers (Wolf et al. 2020). Experiments are conducted on four Nvidia RTX 3090 GPUs.

Inference Following prior work on input-adaptive inference (Teerapittayanon, McDanel, and Kung 2016; Kaya, Hong, and Dumitras 2019), inference is on a per-instance basis, i.e., the batch size for inference is set to 1. This setting is consistent with the common scenario in the industry where individual requests from different users (Schwartz et al. 2020) come at different time points. We report the median performance over five runs with different random seeds.

Overall comparison

We first compare our FasterBERT with the previous SOTA training methods of multi-exit PLMs. Table 1 reports the performance on the seven GLUE benchmark datasets using ALBERT-base as the backbone model. The upper half of the table reports the performances when fine-tuning ALBERT with the last exit. Then the cross-layer average and best performances are reported in the lower rows of Table 1. The following takeaways can be made.

Our FasterBERT outperforms the baselines With the help of block-wise bypasses, our FasterBERT framework consistently outperforms the baseline methods in terms of the average performances across all the intermediate layers and the cross-layer best scores. The performance margins over the baselines are clear, no matter which architecture the block-wise bypasses choose. The experimental results are consistent with our motivation: introducing the block-wise bypasses can help the intermediate transformer layer to concentrate on providing hidden representations, while the bypasses can provide features more suitable for the current layer's exit. In this way, both the cross-layer best and cross-layer average scores can improve. As a direct result, the best layer's score under our FasterBERT framework is comparable to or better than vanilla fine-tuning. Furthermore, even with a single backbone, our FasterBERT achieves higher cross-layer average scores than Single-exits which contains M models of different depths.

Comparisons of different bypass architectures Table 1 also includes the comparisons for different bypass architectures. An interesting observation is that the best performing bypass architecture is the Conv-bypass (best among six of the seven GLUE tasks) instead of the most architectural and computational complex Attn-bypasses. Intuitively, a convolutional layer in the bypass can help extract semantic features different from those provided by the ALBERT back-

[3]In our initial experiments, we find that different divergence measures like Kullback–Leibler divergence and Jensen-Shannon divergence perform comparably. Thus we adopt the one with the most simple form.

[4]https://huggingface.co/bert-base-uncased.

	RTE		MRPC		CoLA		SST-2		QNLI		QQP		MNLI	
ALBERT-base fine-tuning														
	acc		acc/f1		mcc		acc		acc		acc/f1		acc	
ALBERT+linear exit	77.6		89.1		56.5		91.9		88.0		87.3		83.7	
ALBERT+MHA exit	77.4		88.4		56.6		91.8		87.6		87.4		83.5	
Multi-exit ALBERT fine-tuning														
	acc-avg	acc-best	acc/f1-avg	acc/f1-best	mcc-avg	mcc-best	acc-avg	acc-best	acc-avg	acc-best	acc/f1-avg	acc/f1-best	acc-avg	acc-best
Single exits	67.5	77.6	85.4	89.1	44.7	58.3	89.5	92.3	82.9	88.0	83.3	87.3	76.4	83.7
2ST	68.9	77.6	83.0	89.1	43.9	56.5	89.3	91.9	82.5	88.0	81.9	87.3	76.2	83.7
JT	66.8	72.5	84.4	87.9	43.7	53.3	89.2	91.1	82.6	87.3	82.2	87.2	76.0	83.8
JT+MHA exit	66.5	75.9	84.1	87.2	42.6	54.5	88.2	91.5	81.7	87.6	82.3	87.1	76.8	83.2
GradEquil	67.3	77.4	84.2	88.4	43.6	56.1	89.2	91.8	82.4	88.0	82.4	87.0	76.5	83.6
ALT	68.5	77.8	84.6	88.3	44.1	57.3	88.6	90.6	82.3	87.8	81.5	86.8	76.6	82.9
Global Past-Future	68.1	78.4	84.3	88.1	43.6	57.6	89.1	92.3	82.4	87.9	82.4	87.1	75.9	82.9
Sparse-MT	68.3	78.5	83.9	88.2	44.2	57.7	88.9	92.1	82.5	88.2	82.6	87.2	76.5	82.8
FasterBERT														
+ Simple-bypass	70.0	78.0	85.7	89.1	45.7	58.6	89.8	92.2	83.1	88.3	84.1	87.0	77.5	83.2
+ FC-bypass	69.9	75.8	85.6	89.2	45.9	58.3	89.9	92.5	**83.3**	88.2	83.9	87.2	77.4	83.6
+ Conv-bypass	**70.2**	77.9	**86.0**	89.6	**46.4**	58.7	**90.0**	92.4	83.2	88.1	**84.3**	87.2	**77.8**	83.8
+ Attn-bypass	69.8	78.1	85.9	89.7	46.2	58.5	89.9	92.1	83.0	87.9	84.1	86.9	77.5	83.4

Table 1: Experimental results of models with ALBERT backbone on the GLUE benchmark datasets.

bones. Thus the Conv-bypass can help to improve the current exit's performance. Thus in the ablation studies, the default bypass architecture is set as the Conv-bypass.

Demonstrating that FasterBERT does not achieve improvements by merely adding more parameters In the baselines, we also run JT with MHA exits to demonstrate that the improvements of FasterBERT do not come from additional parameters or model structures. Note that the MHA exit consists of a down-projection layer and a self-attn layer with two activation functions. With more parameters and similar architectures, the multi-exit BERT enhanced by MHA exits does not bring clear improvements and makes the model worse in some GLUE tasks.

We also plot the layer-score curves (shown in Figure 2 of the Appendix) for Single-exits, 2ST, JT, and our FasterBERT (with the Conv-bypasses). Our FasterBERT method improves most of the layers compared to the multi-exit ALBERT trained with 2ST or JT, indicating that our method can effectively exploit the potential of the intermediate ALBERT layers.

Dynamic early exiting performances

We compare our DIVE method with the previous best-performing early exiting methods. For the patience-based method (Zhou et al. 2020), early exiting is run on different patience parameters. For the other methods, we run early exiting under different confidence thresholds or patience parameters so that the speedup-performance curves consist of at least 20 points evenly distributed across the interval $(0, 1)$ of speedup ratios. The early exiting hyper-parameters for each method are reported in the Appendix. The speedup-performance curves for the MRPC, SST-2, and QQP tasks are plotted in Figure 2. Due to limited length, the speedup-performance curves for the other four GLUE tasks are reported in Figure 3 of the Appendix.

The following takeaways can also be made from Figure 2:

- With the same backbone model trained with our FasterBERT framework, our DIVE method achieves better speedup-performance trade-offs than the previous SOTA early exiting methods, especially when the speedup ratio is large. In addition, DIVE can achieve 2-3x speedup without any performance degradation.
- The comparison between Patience (with the model trained with FasterBERT) and JT+Patience (with the model trained with the JT method) demonstrates that our FasterBERT can provide superior backbones for early exiting and consistently result in superior performances under different speedup requirements.

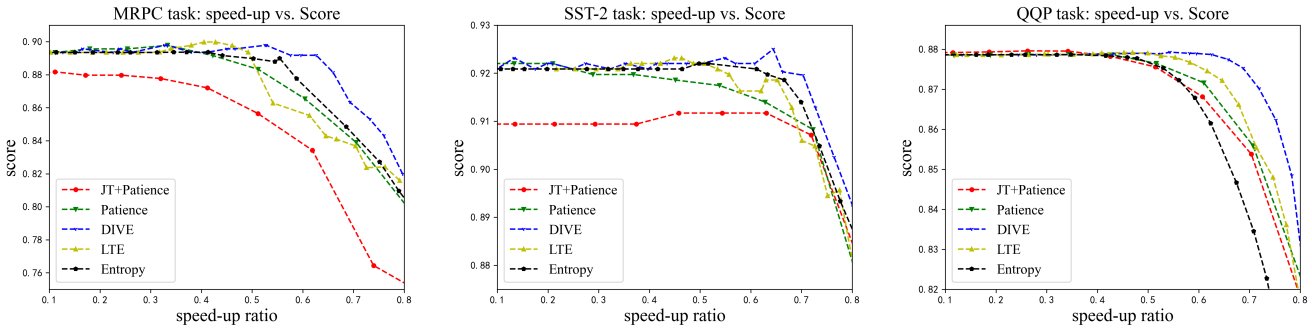
	RTE		SST-2	
	acc-avg	acc-best	acc-avg	acc-best
block-wise bypasses	**70.2**	77.9	**90.0**	92.4
MHSA bypasses	68.4	77.3	89.4	91.7
FFN bypasses	69.6	77.8	89.7	92.1

Table 2: Experimental results for different placement settings for the bypasses.

Analysis and ablation studies

Ablation on the placement of bypasses Since we want to provide separate hidden states in a single forward pass on the BERT backbone, our bypasses must be added to the end of the transformer layer. To show that the design of our block-wise bypasses is essential, we now consider another two placements of bypasses: (a) the bypasses are instead placed around the multi-head self-attention module, denoted as MHSA bypasses; (b) the bypasses are instead placed around the positional feed-forward module, denoted as FFN bypasses. Note that MHSA bypasses can not provide two different hidden states at the end of the transformer block within a single forward pass, thus slowing down inference speed. We conducted this ablation study on the RTE and SST-2 datasets, and the results are reported in Table 2. We can see that: (a) FFN bypasses perform comparably with the block-wise bypasses; (b) MHSA bypasses perform worse than the block-wise bypasses since in this setting, the FFN module in a transformer block still has to conduct multi-task learning.

Ablation on whether to pass $H_i^{'}$ to the next transformer layer The core idea of FasterBERT is to provide different intermediate hidden states for different purposes in a single



(a) MRPC (b) SST-2 (c) QQP

Figure 2: The speedup-score curves with different dynamic early exiting methods, on the MRPC, SST-2 and QQP datasets.

	RTE		SST-2	
	acc-avg	acc-best	acc-avg	acc-best
FasterBERT	**70.2**	77.9	**90.0**	92.4
FasterBERT-v2	66.9	72.8	89.2	91.6

Table 3: Experimental results for the ablation study of whether to pass $H_i^{'}$ to the next layer.

	RTE		SST-2	
	acc-avg	acc-best	acc-avg	acc-best
r = 64	70.2	77.8	89.9	92.1
r = 32	70.1	77.6	90.0	92.2
r = 16	**70.2**	77.9	**90.0**	92.4
r = 8	70.0	78.0	89.8	92.3
r = 4	69.9	77.8	89.7	92.0

Table 4: Experimental results for comparing different bottleneck dimensions. The acc-avg and acc-best scores are reported.

forward pass via our block-wise bypasses. As a sanity check and to demonstrate that our design is valid, we now consider the following setting: passing $H_i^{'}$ to the next layer and also using it for prediction. We will denote this setting as FasterBERT-v2. We use FasterBERT-v2 for training on RTE and SST-2 datasets, and the results are reported in Table 3.

From Table 3, we can see that FasterBERT-v2's performances are significantly worse than FasterBERT (with 1% p-value threshold and Wilcoxon signed-rank test). FasterBERT-v2 asks $H_i^{'}$ to complete two tasks simultaneously, reducing our method to a modified version of the JT method. The results show that our design of providing different representations for different purposes via block-wise bypasses is the key to improving the overall performances of multi-exit ALBERT models.

Comparisons of different bottleneck dimensions r We set the bottleneck dimension r to be 16 for the main part of the experiments. To validate our setting and investigate the effect of different bottleneck dimensions on the model, We conducted experiments on the RTE and SST-2 tasks, and 4 reports the results. From Table 4, we can see that smaller bottleneck dimensions do not result in significant performance drops. An intriguing observation is that larger bottleneck dimensions do not provide performance improvements, demonstrating that the superior performances of our FasterBERT indeed come from our block-wise bypass mechanism instead of the additional parameters.

Ablation on different PLMs To show that our FasterBERT framework is off-the-shelf and can work well with other pre-trained models, we now switch the backbone PLM to BERT-base (Devlin et al. 2019), or ElasticBERT (Liu et al. 2022). The results are reported in Table 3 of the Appendix due to limited length. The results demonstrate that FasterBERT outperforms the JT and ALT baselines under different backbones by clear margins.

Also, note that the cross-layer average scores of BERT-base and ElasticBERT are lower than that of ALBERT-base, showing that ALBERT base is more suitable for early exiting. We hypothesize that the ALBERT model employs a cross-layer parameter sharing strategy. Thus, the representations given by intermediate blocks are more similar to one another, and the performances of its intermediate layers will be close.

Conclusion

In this work, we propose a novel framework, FasterBERT, to improve the early exiting of pre-trained language models. Firstly, we propose to add block-wise bypasses to facilitate the joint training of the intermediate exits, thus improving the overall performance of multi-exit PLM. Second, we propose a novel divergence-based early exiting method, DIVE, which can effectively mine the exiting signals by comparing the predicted distributions of adjacent intermediate layers. DIVE achieves better efficiency-performance trade-offs than the previous SOTA early exiting methods under the same multi-exit backbone. Our FasterBERT is off-the-shelf and can effectively speed up the inferences of PLMs with less performance degradation.

References

Bolukbasi, T.; Wang, J.; Dekel, O.; and Saligrama, V. 2017. Adaptive Neural Networks for Efficient Inference. In *ICML*.

Devlin, J.; Chang, M.-W.; Lee, K.; and Toutanova, K. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 4171–4186. Minneapolis, Minnesota: Association for Computational Linguistics.

Elbayad, M.; Gu, J.; Grave, E.; and Auli, M. 2020. Depth-Adaptive Transformer. *ArXiv*, abs/1910.10073.

Fukushima, K. 2004. Cognitron: A self-organizing multilayered neural network. *Biological Cybernetics*, 20: 121–136.

He, J.; Zhou, C.; Ma, X.; Berg-Kirkpatrick, T.; and Neubig, G. 2021. Towards a Unified View of Parameter-Efficient Transfer Learning. *ArXiv*, abs/2110.04366.

Hendrycks, D.; and Gimpel, K. 2016. Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units. *ArXiv*, abs/1606.08415.

Hinton, G. E.; Vinyals, O.; and Dean, J. 2015. Distilling the Knowledge in a Neural Network. *ArXiv*, abs/1503.02531.

Hochreiter, S.; and Schmidhuber, J. 1997. Long Short-Term Memory. *Neural Comput.*, 9(8): 1735–1780.

Jiao, X.; Yin, Y.; Shang, L.; Jiang, X.; Chen, X.; Li, L.; Wang, F.; and Liu, Q. 2020. TinyBERT: Distilling BERT for Natural Language Understanding. *ArXiv*, abs/1909.10351.

Kaya, Y.; Hong, S.; and Dumitras, T. 2019. Shallow-Deep Networks: Understanding and Mitigating Network Overthinking. In *ICML*.

Kim, Y. 2014. Convolutional Neural Networks for Sentence Classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1746–1751. Doha, Qatar: Association for Computational Linguistics.

Lan, Z.; Chen, M.; Goodman, S.; Gimpel, K.; Sharma, P.; and Soricut, R. 2020. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. *ArXiv*, abs/1909.11942.

Li, H.; Zhang, H.; Qi, X.; Yang, R.; and Huang, G. 2019. Improved Techniques for Training Adaptive Deep Networks. *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 1891–1900.

Li, L.; Lin, Y.; Chen, D.; Ren, S.; Li, P.; Zhou, J.; and Sun, X. 2021. CascadeBERT: Accelerating Inference of Pre-trained Language Models via Calibrated Complete Models Cascade. In *EMNLP*.

Liao, K.; Zhang, Y.; Ren, X.; Su, Q.; Sun, X.; and He, B. 2021. A Global Past-Future Early Exit Method for Accelerating Inference of Pre-trained Language Models. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2013–2023. Online: Association for Computational Linguistics.

Liu, W.; Zhou, P.; Zhao, Z.; Wang, Z.; Deng, H.; and Ju, Q. 2020. FastBERT: a Self-distilling BERT with Adaptive Inference Time. *ArXiv*, abs/2004.02178.

Liu, X.; Sun, T.; He, J.; Wu, J.; Wu, L.; Zhang, X.; Jiang, H.; Cao, Z.; Huang, X.; and Qiu, X. 2022. Towards Efficient NLP: A Standard Evaluation and A Strong Baseline. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 3288–3303. Seattle, United States: Association for Computational Linguistics.

Radford, A.; Wu, J.; Child, R.; Luan, D.; Amodei, D.; Sutskever, I.; et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8): 9.

Ramachandran, P.; Zoph, B.; and Le, Q. V. 2017. Swish: a Self-Gated Activation Function. *arXiv: Neural and Evolutionary Computing*.

Schwartz, R.; Stanovsky, G.; Swayamdipta, S.; Dodge, J.; and Smith, N. A. 2020. The Right Tool for the Job: Matching Model and Instance Complexities. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 6640–6651. Online: Association for Computational Linguistics.

Sun, T.; Shao, Y.; Li, X.; Liu, P.; Yan, H.; Qiu, X.; and Huang, X. 2020. Learning Sparse Sharing Architectures for Multiple Tasks. In *AAAI*.

Sun, T.; Zhou, Y.; Liu, X.; Zhang, X.; Jiang, H.; Cao, Z.; Huang, X.; and Qiu, X. 2021. Early Exiting with Ensemble Internal Classifiers. *ArXiv*, abs/2105.13792.

Tambe, T.; Hooper, C.; Pentecost, L.; Yang, E.-Y.; Donato, M.; Sanh, V.; Rush, A. M.; Brooks, D. M.; and Wei, G.-Y. 2020. EdgeBERT: Optimizing On-Chip Inference for Multi-Task NLP. *ArXiv*, abs/2011.14203.

Teerapittayanon, S.; McDanel, B.; and Kung, H. T. 2016. BranchyNet: Fast inference via early exiting from deep neural networks. *2016 23rd International Conference on Pattern Recognition (ICPR)*, 2464–2469.

Vaswani, A.; Shazeer, N. M.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, L.; and Polosukhin, I. 2017. Attention is All you Need. *ArXiv*, abs/1706.03762.

Wang, A.; Singh, A.; Michael, J.; Hill, F.; Levy, O.; and Bowman, S. R. 2018. GLUE: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461*.

Wolf, T.; Debut, L.; Sanh, V.; Chaumond, J.; Delangue, C.; Moi, A.; Cistac, P.; Rault, T.; Louf, R.; Funtowicz, M.; Davison, J.; Shleifer, S.; von Platen, P.; Ma, C.; Jernite, Y.; Plu, J.; Xu, C.; Scao, T. L.; Gugger, S.; Drame, M.; Lhoest, Q.; and Rush, A. M. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38–45. Online: Association for Computational Linguistics.

Xin, J.; Tang, R.; Lee, J.; Yu, Y.; and Lin, J. 2020. DeeBERT: Dynamic Early Exiting for Accelerating BERT Inference. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2246–2251. Online: Association for Computational Linguistics.

Xin, J.; Tang, R.; Yu, Y.; and Lin, J. 2021. BERxiT: Early Exiting for BERT with Better fine-tuning and extension to regression. In *Proceedings of the 16th conference of the European chapter of the association for computational linguistics: Main Volume*, 91–104.

Xu, C.; Zhou, W.; Ge, T.; Wei, F.; and Zhou, M. 2020. BERT-of-Theseus: Compressing BERT by Progressive Module Replacing. In *EMNLP*.

Yang, Z.; Dai, Z.; Yang, Y.; Carbonell, J.; Salakhutdinov, R.; and Le, Q. V. 2019. XLNet: Generalized Autoregressive Pretraining for Language Understanding. In *NeurIPS*.

Zhang, Z.; Zhu, W.; Zhang, J.; Wang, P.; Jin, R.; and Chung, T.-S. 2022. PCEE-BERT: Accelerating BERT Inference via Patient and Confident Early Exiting. In *Findings of the Association for Computational Linguistics: NAACL 2022*, 327–338. Seattle, United States: Association for Computational Linguistics.

Zhou, W.; Xu, C.; Ge, T.; McAuley, J.; Xu, K.; and Wei, F. 2020. BERT Loses Patience: Fast and Robust Inference with Early Exit. *ArXiv*, abs/2006.04152.

Zhu, W. 2021a. AutoNLU: Architecture Search for Sentence and Cross-sentence Attention Modeling with Redesigned Search Space. In *NLPCC*.

Zhu, W. 2021b. LeeBERT: Learned Early Exit for BERT with cross-level optimization. In *ACL*.

Zhu, W.; Wang, X.; Ni, Y.; and Xie, G. T. 2021. GAML-BERT: Improving BERT Early Exiting by Gradient Aligned Mutual Learning. In *EMNLP*.